

# Traditional Machine Learning versus Large Language Models for Predicting Ship Turnaround Times at Brazilian Ports

E. C. B. Pacheco<sup>1</sup>, J. A. Ramos<sup>1</sup>

Institute of Mathematics and Computer Sciences (ICMC), University of São Paulo, São Carlos, Brazil  
eduardo.pacheco@usp.br, jonnathanalveskz@outlook.com

**Abstract.** Predicting ship turnaround time at ports is an operationally valuable regression problem on heterogeneous, text-rich tabular data. We ask whether Large Language Models (LLMs) can match purpose-built Machine Learning (ML) on this task, using a leakage-controlled pipeline over seven years of Brazilian National Waterway Transportation Agency (ANTAQ) data. On a fixed 200-berthing test set, with 20 seeds and seed-paired statistics (Wilcoxon, Cohen’s  $d$ ), we compare five classical ML models (tabular features plus multilingual sentence embeddings of cargo, origin and destination) against four neural-text regimes that read the same case as text: frontier LLMs (Qwen3-32B, DeepSeek-V3) prompted zero-, 5- and 20-shot; a small generative LLM (Qwen2.5-1.5B) fine-tuned on 100/500/1500 serialized records; and an encoder with a regression head (the same MiniLM backbone used for the ML embeddings) fine-tuned on 100–12000 records. Tuned XGBoost attains  $R^2 \approx 0.73$ . Every prompted and generative-LLM configuration stays at  $R^2 \leq 0$  (no better than the mean), and in-context examples do not help (20-shot  $\approx$  zero-shot). The encoder with a regression head is the only neural-text regime to exceed  $R^2 = 0$ , rising with data (and without saturating) to  $R^2 \approx 0.43$  at 12000 examples and beating the generative LLM at every budget ( $p < 0.01$ ), yet it still trails every ML model. A computational-cost analysis shows the encoder and ML are orders of magnitude cheaper to run than the generative LLM. We conclude that, for this tabular operational-time problem, the predictive signal lies in engineered tabular features that current text-only models do not recover by prompting or light fine-tuning; a regression head helps and scales, but does not close the gap.

CCS Concepts: • **Computing methodologies** → **Supervised learning by regression**; *Natural language processing*.

Keywords: large language models, machine learning, maritime transportation, regression, ship turnaround time, tabular data

## 1. INTRODUCTION

Maritime transport carries roughly 80% of international trade by volume, and bottlenecks in ship turnaround time propagate through the logistics chain. The Brazilian National Waterway Transportation Agency (ANTAQ) publishes operational records of berthings and cargo, and several studies apply machine learning to these data with high headline numbers. Recent work [Pacheco et al. 2026] shows those numbers are inflated by structural leakage when the one-to-many berthing-to-cargo relation is joined without aggregation, and proposes a leakage-controlled pipeline whose realistic performance ( $R^2$  around 0.6–0.7 for operational targets) is our reference.

In parallel, Large Language Models (LLMs) are increasingly proposed as general predictors that ingest a record as text and emit an answer. This raises a concrete question for applied data mining: *on a real, heterogeneous, text-rich tabular regression problem, can text-based models match a purpose-built ML model?* We study ship turnaround time, where each berthing mixes categorical context (port, operation and navigation type, region), numeric cargo attributes (gross weight, containers), and four high-cardinality textual fields (cargo group, containerized cargo group, origin and destination) that

---

This work was carried out at ICMC-USP. The cleaned dataset (2018–2024), serialized prompts, code, per-seed predictions and metrics are released to support reproducibility.

Copyright©2025 Permission to copy without fee all or part of the material printed in KDMiLe is granted provided that the copies are not made or distributed for commercial advantage, and that notice is given that copying is by permission of the Sociedade Brasileira de Computação.

the reference pipeline encodes with multilingual sentence embeddings.

On a single fixed 200-berthing test set and an identical protocol, we compare four families: (i) five classical ML models using tabular features *and* the cargo/route embeddings; (ii) two frontier LLMs (Qwen3-32B, DeepSeek-V3) prompted zero-, 5- and 20-shot; (iii) a small *generative* LLM (Qwen2.5-1.5B) fine-tuned to emit the number from a natural-language serialization of the record (100/500/1500 examples); and (iv) an *encoder with a regression head* — the same MiniLM backbone that produces the ML embeddings — fine-tuned end-to-end on the same serializations (100 to 12000). Every configuration runs with 20 seeds, compared seed-by-seed with paired Wilcoxon tests and Cohen’s  $d$ . We also report training and inference *cost*.

### Contributions.

- A controlled four-family comparison (tuned ML; prompted LLMs at three shot counts; generative small-LLM fine-tuning; encoder-plus-head fine-tuning) for ship-turnaround regression on a leakage-free ANTAQ split, with 20 seeds and seed-paired statistics.
- Evidence that all prompted and generative-LLM regimes stay at  $R^2 \leq 0$  (no gain from 5- or 20-shot), while tuned XGBoost reaches  $R^2 \approx 0.73$ , with every gap significant at  $p < 0.001$  and large effect size.
- A finding that an encoder with a regression head is the *only* text-based regime to exceed  $R^2 = 0$ , scaling with data (without saturating) to  $R^2 \approx 0.43$  and dominating the generative LLM at every budget, yet still trailing all ML models.
- A computational-cost comparison (training and inference time per regime) and a released, reproducible artifact.

## 2. RELATED WORK

Prior machine learning for stay-time prediction at Brazilian ports reports high headline numbers on ANTAQ — e.g.  $R^2 \approx 0.83$  for turnaround regression [Rao et al. 2025] and  $\approx 74\%$  accuracy for stay-time classification [Abreu et al. 2023] — which Pacheco et al. [Pacheco et al. 2026] audit and show to be inflated by structural leakage: a random row-level split places about 97% of test berthings in training, and under a berthing-level split the same model loses most of its  $R^2$ . We adopt their corrected, leakage-controlled pipeline as the ML reference.

LLMs have been applied to non-language tabular tasks. LIFT [Dinh et al. 2022] fine-tunes language models on serialized rows for classification and regression; TabLLM [Hegselmann et al. 2023] studies few-shot tabular classification via textual templates; and LLMTIME [Gruver et al. 2023] casts numeric forecasting as next-token prediction. Encoders with regression heads (e.g. fine-tuned BERT-style models [Reimers and Gurevych 2019]) are a strong text-regression baseline often omitted from LLM-versus-tabular comparisons. Our study holds the test set and protocol fixed across all families and adds both a tuned gradient-boosting baseline with semantic embeddings and an encoder-plus-head trained on the *same* text the LLMs read. Our finding also echoes the broader observation that tree-based models still tend to outperform deep networks on tabular data [Grinsztajn et al. 2022], here extended to text-based LLM predictors and quantified as a data-efficiency gap.

## 3. PROBLEM AND DATA

Let  $\mathcal{B}$  be the set of berthing events. For each  $b \in \mathcal{B}$  we predict two non-negative duration targets (hours): **TOperacao** (cargo operation,  $T_3$ ) and **TAtacado** (total time moored,  $T_2+T_3+T_4$ ), as separate regressions on the same matrix. We use the pipeline of [Pacheco et al. 2026] over ANTAQ 2018–2024 [ANTAQ 2024]: it aggregates cargo to the berthing level (summing numeric attributes, dominant value for homogeneous categoricals, mass-weighted one-hot-sum for heterogeneous ones); imputes with

train-only conditional rules; encodes the four high-cardinality text fields (cargo group, containerized cargo group, origin, destination) with the multilingual encoder **paraphrase-multilingual-MiniLM-L12-v2** [Reimers and Gurevych 2019] (384 dims each, mass-weighted average); applies a semantic outlier filter; and splits 80/20 stratified by region. Absolute dates and partial-time variables are excluded as they reconstruct the targets. We draw a fixed test set of  $N=200$  berthings; the remaining  $\approx 391,000$  form the training set, with  $\text{IDATRACACAO}_{\text{train}} \cap \text{IDATRACACAO}_{\text{test}} = \emptyset$ . The *same* 200 cases are scored by all systems.

#### 4. EXPERIMENTAL DESIGN

We report RMSE,  $R^2$  and MAE; 20 seeds per configuration.

##### 4.1 Regime 1 — Traditional ML with embeddings

Five models on the full berthing-level matrix (ordinal-encoded categoricals, median target encoding for the mooring port, numeric and mass-weighted one-hot-sum cargo features, *and* the four 384-dim embeddings; 1,606 columns): XGBoost (tuned per target with Optuna), a two-layer MLP,  $k$ -NN ( $k=10$ ), linear regression, and a linear SVM (SGD). Deterministic models ( $k$ -NN) replicate predictions across seeds.

##### 4.2 Regime 2 — Prompted LLMs

Qwen3-32B and DeepSeek-V3 via Amazon Bedrock at temperature 0.1. Each berthing is a compact JSON of all non-target fields; the model returns one number. We evaluate *zero-shot*, *5-shot* and *20-shot* (exemplars sampled per seed). Unparseable replies fall back to the training median (rare).

##### 4.3 Regime 3 — Generative small-LLM fine-tuning

We fine-tune the base (non-instruct) Qwen2.5-1.5B [Qwen Team 2025] with LoRA [Hu et al. 2022] ( $r=16$ ) in **bf16**. Each berthing is first turned into a fluent Portuguese description (e.g. “*cargo of 26,503 tonnes, mainly mineral fuels, by cabotage from Santos to Suape, in January 2019*”) generated deterministically by DeepSeek-V3 and cached; targets are never included. The model learns to emit the number (loss masked to the answer). Conditions: untrained *base*, and fine-tuning on 100/500/1500 serialized records, reseeded per run.

##### 4.4 Regime 4 — Encoder with a regression head

We fine-tune the *same* multilingual MiniLM backbone that produces the ML embeddings, with a two-output linear regression head (joint [T0peracao, TAtacado]), end-to-end on the same serialized descriptions, with standardized targets. Conditions: *base* (untrained head) and fine-tuning on 100 to 12000 examples, 20 seeds.

##### 4.5 Statistical protocol

Each configuration yields 20 per-seed metric values. Pairs of configurations are compared on RMSE seed-by-seed with the paired Wilcoxon signed-rank test [Wilcoxon 1945]; effect size is Cohen’s  $d$  [Cohen 1988] (we report the pooled-SD  $d$ ;  $|d|=0.8$  is large). The test set is fixed, so seeds affect only training; ML uses seeds 0–19 and the text regimes a common 20-seed list, paired by rank.

Table I. Test performance (200 berthings, 20 seeds): RMSE in hours (mean  $\pm$  std) and mean  $R^2$ .

|                                              | TOperacao        |               | TAtacado         |               |
|----------------------------------------------|------------------|---------------|------------------|---------------|
|                                              | RMSE             | $R^2$         | RMSE             | $R^2$         |
| <i>Traditional ML (tabular + embeddings)</i> |                  |               |                  |               |
| XGBoost                                      | 19.10 $\pm$ 0.17 | +0.725        | 20.96 $\pm$ 0.17 | +0.728        |
| MLP                                          | 20.57 $\pm$ 0.51 | +0.681        | 23.09 $\pm$ 0.63 | +0.670        |
| KNN                                          | 22.17 $\pm$ 0.00 | +0.629        | 23.12 $\pm$ 0.00 | +0.669        |
| Linear Reg.                                  | 25.15 $\pm$ 0.22 | +0.523        | 27.72 $\pm$ 0.22 | +0.525        |
| SVM                                          | 26.30 $\pm$ 0.07 | +0.478        | 28.68 $\pm$ 0.08 | +0.491        |
| <i>Encoder + regression head (MiniLM)</i>    |                  |               |                  |               |
| base                                         | 36.53 $\pm$ 0.00 | -0.007        | 40.30 $\pm$ 0.00 | -0.005        |
| FT-100                                       | 35.16 $\pm$ 1.19 | +0.066        | 39.16 $\pm$ 1.49 | +0.050        |
| FT-500                                       | 33.77 $\pm$ 1.32 | +0.138        | 37.75 $\pm$ 1.49 | +0.117        |
| FT-1500                                      | 30.89 $\pm$ 0.96 | +0.279        | 34.03 $\pm$ 1.01 | +0.283        |
| FT-3000                                      | 29.39 $\pm$ 0.92 | +0.348        | 32.86 $\pm$ 1.10 | +0.331        |
| FT-6000                                      | 28.57 $\pm$ 0.91 | +0.384        | 31.23 $\pm$ 0.97 | +0.396        |
| FT-12000                                     | 27.87 $\pm$ 0.53 | <b>+0.414</b> | 30.37 $\pm$ 0.71 | <b>+0.429</b> |
| <i>Generative small LLM (Qwen2.5-1.5B)</i>   |                  |               |                  |               |
| base                                         | 217.0 $\pm$ 0.0  | -34.54        | 217.7 $\pm$ 0.0  | -28.33        |
| FT-100                                       | 38.31 $\pm$ 3.29 | -0.115        | 49.29 $\pm$ 15.1 | -0.636        |
| FT-500                                       | 36.40 $\pm$ 2.59 | -0.005        | 41.05 $\pm$ 3.02 | -0.048        |
| FT-1500                                      | 36.15 $\pm$ 2.66 | +0.009        | 40.11 $\pm$ 2.49 | +0.001        |
| FT-3000                                      | 36.58 $\pm$ 1.98 | -0.012        | 40.64 $\pm$ 3.08 | -0.027        |
| FT-6000                                      | 34.52 $\pm$ 2.38 | +0.097        | 37.80 $\pm$ 2.14 | +0.113        |
| <i>Prompted frontier LLMs</i>                |                  |               |                  |               |
| Qwen3-32B 0-shot                             | 36.48 $\pm$ 0.27 | -0.004        | 40.39 $\pm$ 0.56 | -0.009        |
| Qwen3-32B 5-shot                             | 36.55 $\pm$ 2.73 | -0.013        | 41.20 $\pm$ 3.86 | -0.059        |
| Qwen3-32B 20-shot                            | 36.25 $\pm$ 2.85 | +0.003        | 41.50 $\pm$ 4.71 | -0.079        |
| DeepSeek-V3 0-shot                           | 36.45 $\pm$ 0.33 | -0.002        | 42.03 $\pm$ 0.20 | -0.093        |
| DeepSeek-V3 5-shot                           | 44.23 $\pm$ 18.9 | -0.732        | 44.69 $\pm$ 11.5 | -0.313        |
| DeepSeek-V3 20-shot                          | 38.18 $\pm$ 6.22 | -0.128        | 43.34 $\pm$ 9.41 | -0.214        |

## 5. RESULTS

Table I reports all configurations; Table II compares XGBoost against each text-based configuration; Figure 1 shows the data-efficiency curves (with a data-matched XGBoost curve).

### 5.1 ML versus text-based models

Every ML model outperforms every text-based configuration on both targets. Table II compares XGBoost against the strongest configuration of each text family: all gaps are significant at  $p < 0.001$  with large pooled- $d$ , and the RMSE deficit is 11–28 hours. Even the weakest ML model (SVM,  $R^2 \approx 0.48$ ) beats the best text-based model.

### 5.2 Prompting and fine-tuning data-efficiency

*In-context examples do not help.* For both frontier models, 5-shot and 20-shot are statistically indistinguishable from zero-shot (all  $p > 0.1$ ); 20-shot  $\approx$  zero-shot and few-shot can destabilize DeepSeek-V3 (std up to 19 h). *Generative fine-tuning barely moves above  $R^2 \approx 0$ :* it rescues the small model from an unusable base ( $R^2 \approx -30$ ) but stays at  $R^2 \approx 0$  through 3000 examples and only edges up to  $\approx 0.10$  at 6000 — at or below the level of a 32B model prompted zero-shot. *The encoder head is the exception:* its untrained base behaves as a mean predictor ( $R^2 \approx 0$ ), and fine-tuning lifts  $R^2$  monotonically and *without saturating* across 100/500/1500/3000/6000/12000 examples

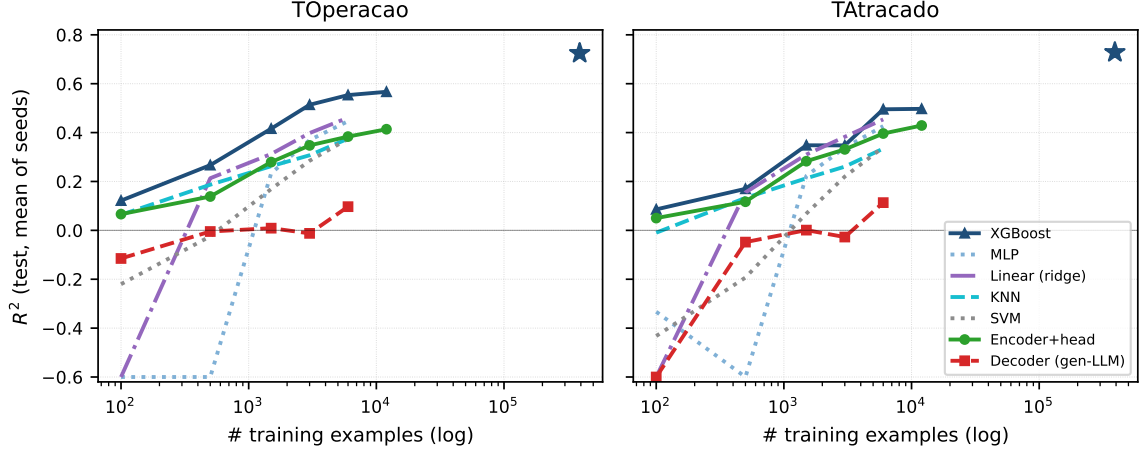


Fig. 1. Data-matched data-efficiency ( $R^2$  vs. number of training examples, log scale) for all seven model families trained on the *same*  $N$  random berthings (100–6000, 20 seeds);  $\star$  is XGBoost’s full-data point ( $N=391,460$ ). XGBoost leads at every budget. The text encoder catches KNN/SVM by  $N=6000$ ; linear and MLP overfit catastrophically at small  $N$  in the 1606-dim feature space (clipped at  $-0.6$ ); the generative decoder stays near 0 until a small rise at 6000.

Table II. Paired comparison of XGBoost vs. each text-based configuration.  $\Delta\text{RMSE} = \text{mean}(\text{config}) - \text{mean}(\text{XGBoost})$  in hours (positive = XGBoost better);  $d$  is pooled Cohen’s  $d$ ; significance from paired Wilcoxon ( $*p<0.05$ ,  $**p<0.01$ ,  $***p<0.001$ ).

| vs. XGBoost        | TOperacao           |          | TAtacado            |           |
|--------------------|---------------------|----------|---------------------|-----------|
|                    | $\Delta\text{RMSE}$ | $d$      | $\Delta\text{RMSE}$ | $d$       |
| Encoder FT-12000   | +8.8                | -21.6*** | +9.4                | -17.8***  |
| Encoder FT-3000    | +10.3               | -15.2*** | +11.9               | -14.8***  |
| Encoder FT-100     | +16.1               | -19.0*** | +18.2               | -17.2***  |
| Gen-LLM FT-1500    | +17.1               | -9.0***  | +19.2               | -10.9***  |
| Gen-LLM FT-500     | +17.3               | -9.4***  | +20.1               | -9.4***   |
| Gen-LLM FT-100     | +19.2               | -8.3***  | +28.3               | -2.7***   |
| Qwen3-32B 0-shot   | +17.4               | -76.7*** | +19.4               | -46.9***  |
| Qwen3-32B 20-shot  | +17.2               | -8.5***  | +20.5               | -6.2***   |
| DeepSeek-V3 0-shot | +17.3               | -66.2*** | +21.1               | -116.1*** |

(0.07  $\rightarrow$  0.14  $\rightarrow$  0.28  $\rightarrow$  0.35  $\rightarrow$  0.38  $\rightarrow$  0.41 on TOperacao; 0.05  $\rightarrow$  0.12  $\rightarrow$  0.28  $\rightarrow$  0.33  $\rightarrow$  0.40  $\rightarrow$  0.43 on TAtacado), beating the generative LLM at every budget (paired  $p < 0.01$ ; pooled  $d$  from  $-0.95$  to  $-3.2$ ). Every doubling helps significantly — even the 6000  $\rightarrow$  12000 step ( $p < 0.05$ ,  $d \approx 0.9$ ) — so the curve has not flattened. At 12000 examples ( $R^2 \approx 0.41$ –0.43) the encoder approaches the weakest ML models (SVM/linear,  $R^2 \approx 0.48$ –0.52), yet remains well below XGBoost’s 0.73, and the marginal gains are shrinking.

*Data-matched comparison.* To separate representation from data budget, we train *all seven families* on the same subsample sizes (100–6000 random berthings, 20 seeds, identical features for the ML models; Figure 1, Table III). Four findings emerge. (i) XGBoost leads at *every* budget — already at  $N=100$  ( $R^2$  0.12 vs. the encoder’s 0.07) — and is the most data-efficient: it reaches the encoder’s  $N=12000$  level ( $R^2 \approx 0.42$ ) with only  $\approx 1500$  examples and trains 10–50 $\times$  faster (Table IV). (ii) At small  $N$  the high-dimensional (1606-feature) space breaks the linear and MLP models, whose unregularized/over-parameterized fits give large negative  $R^2$  at  $N \leq 500$ ; trees, KNN and the encoder stay robust. (iii) The text *encoder* is competitive with standard ML at matched data — by  $N=6000$  it ( $R^2 \approx 0.38$ –0.40) matches KNN and SVM ( $\approx 0.34$ –0.38), trailing only XGBoost, MLP and ridge.

Table III. Data-matched test  $R^2$  (mean over 20 seeds) by training-set size  $N$ , all families. Negative values ( $< -1$ ) truncated. ML models share the engineered feature matrix; encoder/decoder read the serialized text.

|                   | $N$ (training examples) |              |              |              |              |
|-------------------|-------------------------|--------------|--------------|--------------|--------------|
|                   | 100                     | 500          | 1500         | 3000         | 6000         |
| <b>TOperacao</b>  |                         |              |              |              |              |
| XGBoost           | <b>0.122</b>            | <b>0.267</b> | <b>0.416</b> | <b>0.514</b> | <b>0.553</b> |
| MLP               | –                       | –            | 0.233        | 0.369        | 0.446        |
| Linear (ridge)    | –                       | 0.213        | 0.312        | 0.398        | 0.461        |
| KNN               | 0.064                   | 0.187        | 0.262        | 0.307        | 0.375        |
| SVM               | -0.22                   | -0.02        | 0.169        | 0.285        | 0.374        |
| Encoder           | 0.066                   | 0.138        | 0.279        | 0.348        | 0.384        |
| Decoder (LLM)     | -0.12                   | -0.01        | 0.009        | -0.01        | 0.097        |
| <b>TAttracado</b> |                         |              |              |              |              |
| XGBoost           | <b>0.086</b>            | <b>0.170</b> | <b>0.348</b> | <b>0.348</b> | <b>0.495</b> |
| MLP               | -0.33                   | –            | 0.223        | 0.338        | 0.426        |
| Linear (ridge)    | -0.81                   | 0.155        | 0.310        | 0.383        | 0.454        |
| KNN               | -0.01                   | 0.133        | 0.213        | 0.261        | 0.334        |
| SVM               | -0.43                   | -0.19        | 0.069        | 0.220        | 0.340        |
| Encoder           | 0.050                   | 0.117        | 0.283        | 0.331        | 0.396        |
| Decoder (LLM)     | -0.64                   | -0.05        | 0.001        | -0.03        | 0.113        |

Table IV. Computational cost. Training is one-time (per seed for fine-tuning); inference is for the full 200-case test set unless noted. ML on CPU; encoder/generative LLM on M5 Pro (MPS); prompted LLMs via Bedrock.

| Regime                | Training               | Inference (200 cases)     |
|-----------------------|------------------------|---------------------------|
| XGBoost (ML)          | 182 s (+9 s embed.)    | 3 ms                      |
| Encoder + head        | 33–380 s (100–12000)   | $\approx 0.9$ s           |
| Generative LLM (1.5B) | 600–1170 s (3000–6000) | $\approx 20$ s            |
| Prompted LLM (32B)    | none                   | 0.6–2.0 s <i>per case</i> |

(iv) The generative *decoder* stays at  $R^2 \approx 0$  through 3000 and only edges up at 6000 ( $\approx 0.10$ ). The full ML model uses all 391,460 berthings ( $\star$ ); the largest data-matched budget here (6000) is 1.5% of that, so the residual XGBoost-vs-encoder gap is both representational and one of budget. A log-linear fit of the encoder’s  $R^2$  against  $\ln N$  extrapolates that matching XGBoost’s full-data  $R^2 \approx 0.73$  would take  $\sim 4 \times 10^5$  serialized examples (essentially the entire training set), while reaching the weakest ML tier ( $R^2 \approx 0.5$ ) needs  $\sim 25,000$  ( $\approx 2 \times$  our largest budget) — and these are optimistic bounds, since the curve is already bending.

### 5.3 Computational cost

Table IV contrasts training and inference cost (local Apple M5 Pro for ML, encoder and generative LLM; Bedrock API for prompted LLMs; absolute values are not directly comparable across local vs. API, but orders of magnitude are). The encoder is the cheapest learned model (train 33–380 s, inference  $< 1$  s for all 200 cases) and ML inference is sub-second; the generative LLM is by far the most expensive (train  $\approx 10$ –20 min at 3000–6000 examples,  $\approx 20$  s of autoregressive decoding for 200 cases) while being *less* accurate, and prompted LLMs require an API round-trip per case that grows with the number of in-context examples. At the largest data-matched budget ( $N=6000$ ) the contrast is stark: training takes  $\approx 4$  s (XGBoost),  $\approx 290$  s (encoder) and  $\approx 1170$  s (generative LLM), while inference over the 200-case test set takes  $\approx 1$  ms,  $\approx 0.9$  s and  $\approx 20$  s respectively.

## 6. DISCUSSION

Three points stand out. First, *representation, not model size, is decisive*: a 1.5B model fine-tuned on 1500 examples and a 32B model prompted with 20 examples both stall at  $R^2 \approx 0$ , while the small encoder reading the same text reaches 0.43 (still rising at 12000 examples) and tuned trees reach 0.73 — the gains come from how cargo/route information is represented (dense embeddings, mass-weighted mixtures, target encoding) rather than from scale or prompting. Tellingly, XGBoost consumes the *same* MiniLM embeddings *frozen* (as features), whereas the encoder *fine-tunes* that very backbone end-to-end with a regression head — yet frozen-embeddings-plus-trees wins at every budget, indicating that the engineered tabular features and the tree’s nonlinear interactions matter more than end-to-end adaptation of the text encoder. Second, *a regression head matters and scales*: turning the task from text generation into supervised regression converts an LLM-family backbone from  $R^2 \leq 0$  into a model that improves monotonically with data, suggesting further gains with more examples — but it remains text-only and does not see the engineered tabular features. Third, *cost compounds the accuracy gap*: the most expensive learned model (generative LLM) is also the least accurate, whereas ML and the encoder are cheap to train and near-instant at inference.

**Limitations.** The comparison is deliberately asymmetric in representation: ML uses engineered features and embeddings, the text regimes read raw fields as text; this reflects how each family is used in practice. The test set has 200 fixed, leakage-free cases; seed sets differ across families and are paired by rank. We explored one small generative model and one encoder backbone under modest budgets; larger backbones, more examples, or hybrid text-plus-tabular models could narrow the gap and are left to future work.

**Practical implications and future work.** For heterogeneous operational-time prediction with a documented schema and abundant records, our results recommend tuned gradient boosting over semantic embeddings: it is both the most accurate and, at milliseconds per inference and minutes to train, by far the cheapest. Text-based models become attractive mainly in regimes this study does not target — very low data, undocumented or shifting schemas, or zero-infrastructure settings — where reading raw records as text sidesteps feature engineering. Promising directions include hybrid models that fuse the engineered tabular features with a fine-tuned text encoder, larger or instruction-tuned encoder backbones, and serializing the full training set to test whether the encoder’s still-rising curve ultimately meets the tabular ceiling.

## 7. CONCLUSION

On a leakage-controlled seven-year ANTAQ benchmark for ship turnaround time, traditional ML with semantic embeddings decisively outperforms text-based models across prompting and fine-tuning: tuned XGBoost reaches  $R^2 \approx 0.73$  while no prompted or generative-LLM configuration beats a mean predictor, and every gap is significant at  $p < 0.001$  with large effect size. An encoder with a regression head is the only text regime to extract real signal ( $R^2$  up to 0.43 and still rising at 12000 examples) and dominates the generative LLM at equal data and far lower cost, yet still trails all ML models. For heterogeneous operational-time regression of this kind, our evidence favors purpose-built ML with engineered semantic features over text-only models used as direct predictors.

## 8. CODE AND DATA

All code, serialized prompts, per-seed predictions, metrics and the figures of this paper are released at <https://github.com/eduardocbpacheco/antaq-ml-vs-llm> [Pacheco and Ramos 2026]. The underlying cleaned ANTAQ dataset (2018–2024) and the leakage-controlled processing pipeline are available on Zenodo (DOI 10.5281/zenodo.20549160) [Pacheco and Louzada 2026], and the corrected pipeline is detailed in [Pacheco et al. 2026].

## ACKNOWLEDGMENT

The authors thank ICMC–USP for computational support.

## REFERENCES

- ABREU, L. R., MACIEL, I. S. F., ALVES, J. S., BRAGA, L. C., AND PONTES, H. L. J. A decision tree model for the prediction of the stay time of ships in Brazilian ports. *Engineering Applications of Artificial Intelligence* 117 (1): 105634, 2023.
- ANTAQ. Estatístico Aquaviário — Agência Nacional de Transportes Aquaviários. <https://web.antaq.gov.br/anuario/>, 2024.
- COHEN, J. *Statistical Power Analysis for the Behavioral Sciences*. Lawrence Erlbaum Associates, Hillsdale, USA, 1988.
- DINH, T., ZENG, Y., ZHANG, R., LIN, Z., GIRA, M., RAJPUT, S., YONG SOHN, J., PAPAILIOPOULOS, D., AND LEE, K. LIFT: Language-Interfaced Fine-Tuning for Non-Language Machine Learning Tasks. In *Advances in Neural Information Processing Systems*. New Orleans, USA, pp. 11763–11784, 2022.
- GRINSZTAJN, L., OYALLON, E., AND VAROQUAUX, G. Why do tree-based models still outperform deep learning on typical tabular data? In *Advances in Neural Information Processing Systems*. New Orleans, USA, pp. 507–520, 2022.
- GRUVER, N., FINZI, M., QIU, S., AND WILSON, A. G. Large Language Models Are Zero-Shot Time Series Forecasters. In *Advances in Neural Information Processing Systems*. New Orleans, USA, pp. 19622–19635, 2023.
- HEGSELMANN, S., BUENDIA, A., LANG, H., AGRAWAL, M., JIANG, X., AND SONTAG, D. TabLLM: Few-shot Classification of Tabular Data with Large Language Models. In *Proceedings of the International Conference on Artificial Intelligence and Statistics*. Valencia, Spain, pp. 5549–5581, 2023.
- HU, E. J., SHEN, Y., WALLIS, P., ALLEN-ZHU, Z., LI, Y., WANG, S., WANG, L., AND CHEN, W. LoRA: Low-Rank Adaptation of Large Language Models. In *Proceedings of the International Conference on Learning Representations*. Online, pp. 1–13, 2022.
- PACHECO, E. C. B. AND LOUZADA, F. ANTAQ Ship-Turnaround Dataset and Reproduction Artifacts. Zenodo, <https://doi.org/10.5281/zenodo.20549160>, 2026.
- PACHECO, E. C. B., MARTINS, R. P., GUALBERTO, A. S., GERMANO, J. V. S., NETO, M. M., AND LOUZADA, F. Methodological Pitfalls in Predicting Ship Turnaround Time at Brazilian Ports: An Empirical Audit and a Reproducible Pipeline. Manuscript under review, 2026.
- PACHECO, E. C. B. AND RAMOS, J. A. antaq-ml-vs-llm: code, metrics and predictions. <https://github.com/eduardocbpacheco/antaq-ml-vs-llm>, 2026.
- QWEN TEAM. Qwen2.5 Technical Report. arXiv:2412.15115, 2025.
- RAO, A. R., WANG, H., AND GUPTA, C. Predictive Analysis for Optimizing Port Operations. *Applied Sciences* 15 (6): 2877, 2025.
- REIMERS, N. AND GUREVYCH, I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. Hong Kong, China, pp. 3982–3992, 2019.
- WILCOXON, F. Individual Comparisons by Ranking Methods. *Biometrics Bulletin* 1 (6): 80–83, 1945.